WEB PROGRAMMING

Benoit Gaudou
benoit.gaudou@ut-capitole.fr

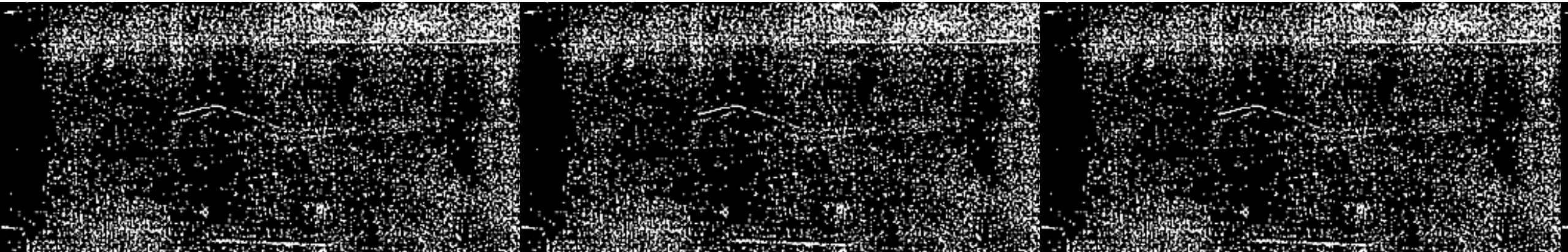
UNIVERSITÉ TOULOUSE 1 CAPITOLE

WEB 编程

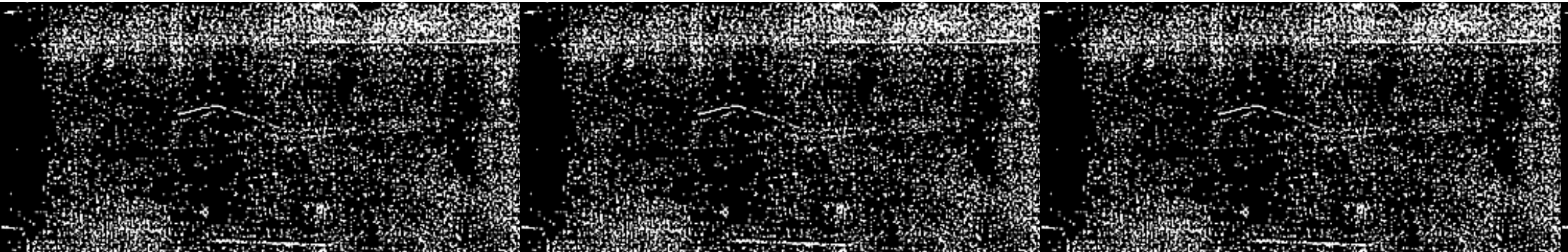
贝努瓦·高杜
benoit.gaudou@ut-capitole.fr

UNIVERSITÉ TOULOUSE 1 CAPITOLE

OPENAPI SPECIFICATION



OPENAPI SPECIFICATION



OPENAPI

- La spécification OpenAPI (anciennement Swagger Specification) est un format de description de l'API des web services RESTful.
- La specification se concentre sur la définition des différentes routes avec des exemples d'effets/valeurs retournées.
- La spécification est langage agnostique, dans un format simple et doté d'outils pour une visualisation graphique.

OPENAPI

- **OpenAPI规范（原名Swagger Specification）是一种用于描述RESTful网络服务API的格式标准。**
- 该规范着重于各类路径的定义，并通过示例展示其效果/返回值。
- 该规范采用语言无关性设计，采用简洁格式并配备图形可视化工具。

HELLOWORLD OPENAPI

```yaml
openapi: 3.0.2
info:
  version: '1.0'
  title: Hello world API
servers:
  - url: http://localhost:8080/
paths:
  /hello:
    get:
      summary: Returns a Helloworld message
      description: optional description
      responses:
        '200':
         description: OK
        content:
          text/plain:
            schema:
              type: string
              example: "Hello Roy !"
```

HELLOWORLD OPENAPI

```
openapi :3.0.2 信
息：
  版本：  '1.0'
  title: Hello world API服务器:
  - 网址: http://localhost:8080/
路径：
  / 你好:
    获取:
      summary: 返回一个Helloworld消息
      描述: 可选描述回答:
        '200':
          描述: OK
          内容:
            正文：
              模式：
                类型: 字符串
                例如: "你好，罗伊！"
```

VISUALISATION

Swagger Editor
Supported by SMARTBEAR

File ▾ Edit ▾ Insert ▾ Generate Server ▾ Generate Client ▾

```yaml
openapi: 3.0.2
info:
  version: '1.0'
  title: Hello world API
servers:
  - url: http://localhost:8080/
paths:
  /hello:
    get:
      summary: Returns a HelloWorld.
      description: Optional description of the HelloWorld.
      responses:
        '200':
          description: OK
          content:
            text/plain:
              schema:
                type: string
                example: "Hello Roy !"
```

Hello world API 1.0 OAS3

Servers

http://localhost:8080/

default

GET /hello Returns a HelloWorld.

VISUALISATION

Swagger Editor
Supported by SMARTBEAR

File ▾ Edit ▾ Insert ▾ Generate Server ▾ Generate Client ▾

```yaml
openapi: 3.0.2
info:
  version: '1.0'
  title: Hello world API
servers:
  - url: http://localhost:8080/
paths:
  /hello:
    get:
      summary: Returns a HelloWorld.
      description: Optional description of the HelloWorld.
      responses:
        '200':
          description: OK
          content:
            text/plain:
              schema:
                type: string
                example: "Hello Roy !"
```

Hello world API 1.0 OAS3

Servers

http://localhost:8080/

default

GET /hello Returns a HelloWorld.

STRUCTURE DE BASE

- **metadata** : version openapi, infos
- **servers** : URL de base du server
- **paths** : décris l'ensembles des routes (opérations) proposées par l'API / le web service
  - Méthode HTTP
  - paramètres
  - request body
  - response : code de retour, réponse

Swagger Editor
Supported by SMARTBEAR
File ▾ Edit ▾ Insert ▾ Generate Server ▾ Gener

```yaml
openapi: 3.0.2
info:
  version: '1.0'
  title: Hello world API
servers:
  - url: http://localhost:8080/
paths:
  /hello:
    get:
      summary: Returns a HelloWorld.
      description: Optional description of the HelloWorld.
      responses:
        '200':
          description: OK
          content:
            text/plain:
              schema:
                type: string
                example: "Hello Roy !"
```

https://swagger.io/docs/specification/basic-structure/

STRUCTURE DE BASE

- 元数据：版本 openapi，信息
- 服务器：服务器的主URL
- 路径：描述API/网络服务所提供的路由（操作）集合
  - HTTP方法
  - 参数
  - 请求体
  - 响应：返回代码，响应

Swagger Editor Supported by SMARTBEAR File ▾ Edit ▾ Insert ▾ Generate Server ▾ Gener

```yaml
openapi: 3.0.2
info:
  version: '1.0'
  title: Hello world API
servers:
  - url: http://localhost:8080/
paths:
  /hello:
    get:
      summary: Returns a HelloWorld.
      description: Optional description of the HelloWorld.
      responses:
        '200':
          description: OK
          content:
            text/plain:
              schema:
                type: string
                example: "Hello Roy !"
```

https://swagger.io/docs/specification/basic-structure/

PARAMÈTRES

- Paramètres de la requête :
  - Dans l'URL (equivalent aux PathVariable) (/users/{userId}),
    - Défini comme **parameters in path**

```yaml
paths:
  /users/{userId}:
    get:
      summary: Returns a user by ID.
      parameters:
        - name: userId
          in: path
          required: true
          description: Parameter description
          schema:
            type : integer
            format: int64
            minimum: 1
      responses:
        '200':
          description: OK
```

PARAMÈTRES

- 查询参数：
  - 在URL中（相当于PathVariable）(/users/{userId}），
    - 定义为路径中的参数

```yaml
paths:
  /users/{userId}:
    get:
      summary: Returns a user by ID.
      parameters:
        - name: userId
          in: path
          required: true
          description: Parameter description
          schema:
            type : integer
            format: int64
            minimum: 1
      responses:
        '200':
          description: OK
```

PARAMÈTRES

- Paramètres de la requête :
  - Paramètres dans l'URL (/users/34/?userId=admin),
    - Défini comme un parameters in query

```yaml
openapi: 3.0.2
info:
  version: '1.0'
  title: Hello world API
servers:
  - url: http://localhost:8080/
paths:
  /hello:
    get:
      summary: Returns a HelloWorld.
      description: Optional description of the HelloWorld.
      parameters:
        - name: name
          in: query
          required: false
          schema:
            type: string
      responses:
        '200':
          description: OK
          content:
            text/plain:
              schema:
                type: string
                example: "Hello Roy !"
```

PARAMÈTRES

- 查询参数：
  - URL 中 的 参 数 （/users/34/?userId=admin）
    - 定义为查询中的参数

```yaml
openapi: 3.0.2
info:
  version: '1.0'
  title: Hello world API
servers:
  - url: http://localhost:8080/
paths:
  /hello:
    get:
      summary: Returns a HelloWorld.
      description: Optional description of the HelloWorld.
      parameters:
        - name: name
          in: query
          required: false
          schema:
            type: string
      responses:
        '200':
          description: OK
          content:
            text/plain:
              schema:
                type: string
                example: "Hello Roy !"
```

REQUEST BODY

- Avec les requêtes POST (en particulier), on doit définir les données passées dans le body de la reqête HTTP.

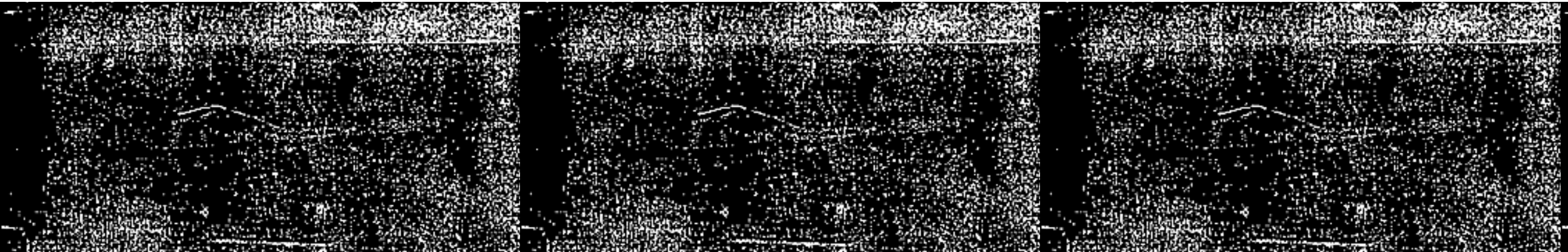
```yaml
paths:
  /users:
    post:
      summary: Creates a user.
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                username:
                  type: string
      responses:
        '201':
          description: Created
```

REQUEST BODY

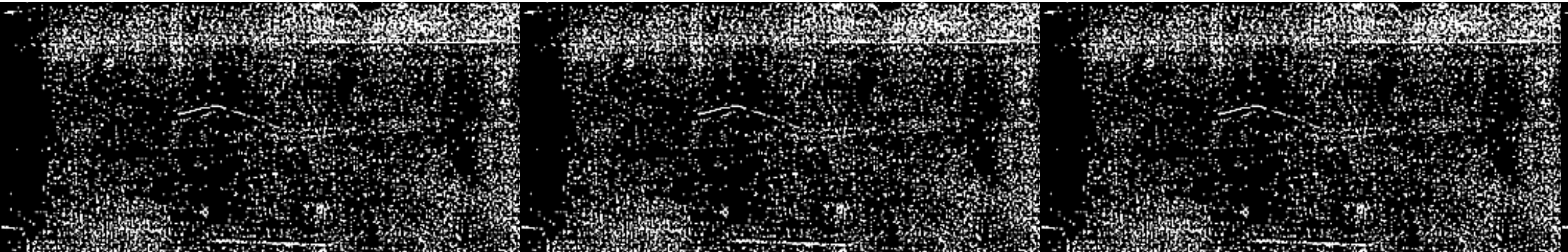
- 在使用POST请求（尤其是POST请求）时，必须在HTTP请求的body部分定义所传递的数据。

```yaml
paths:
  /users:
    post:
      summary: Creates a user.
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                username:
                  type: string
      responses:
        '201':
          description: Created
```

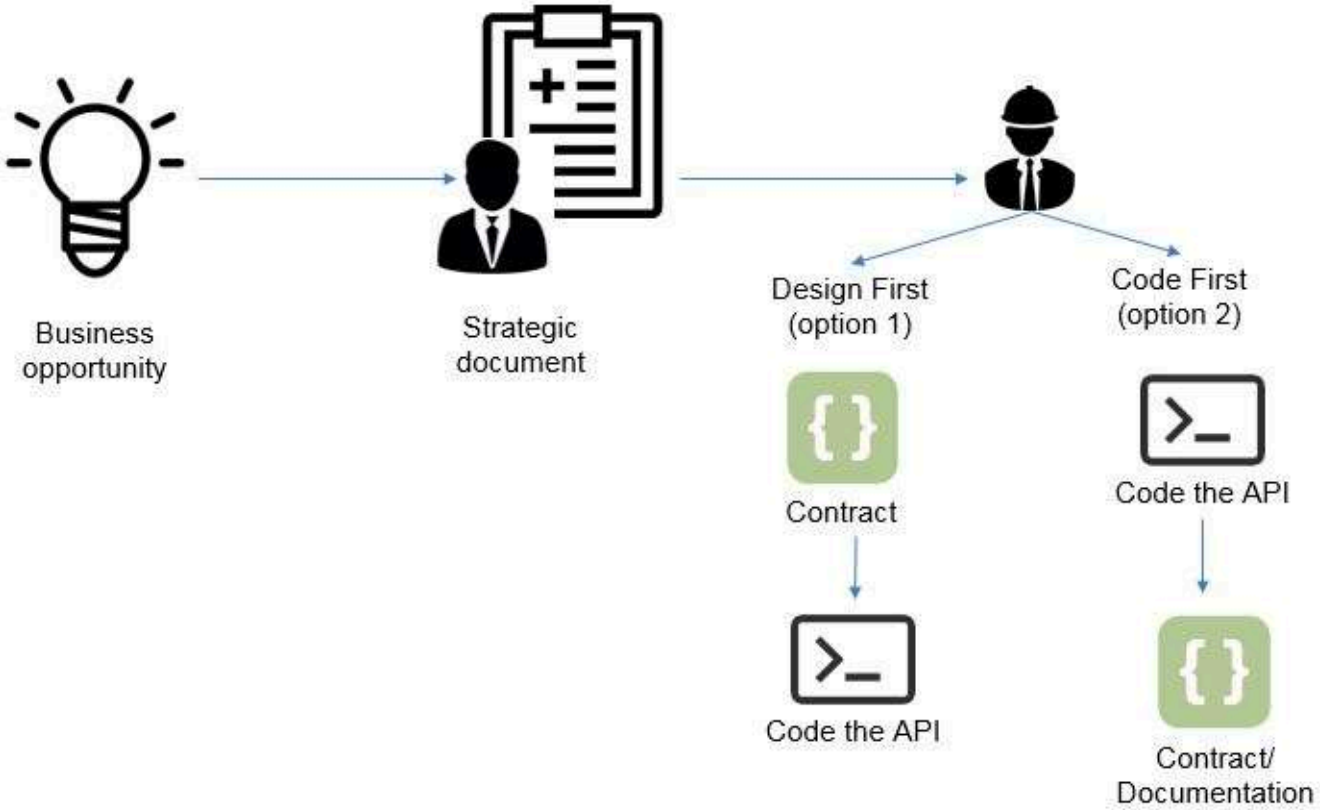
DESIGN FIRST VS CODE FIRST



DESIGN FIRST VS CODE FIRST

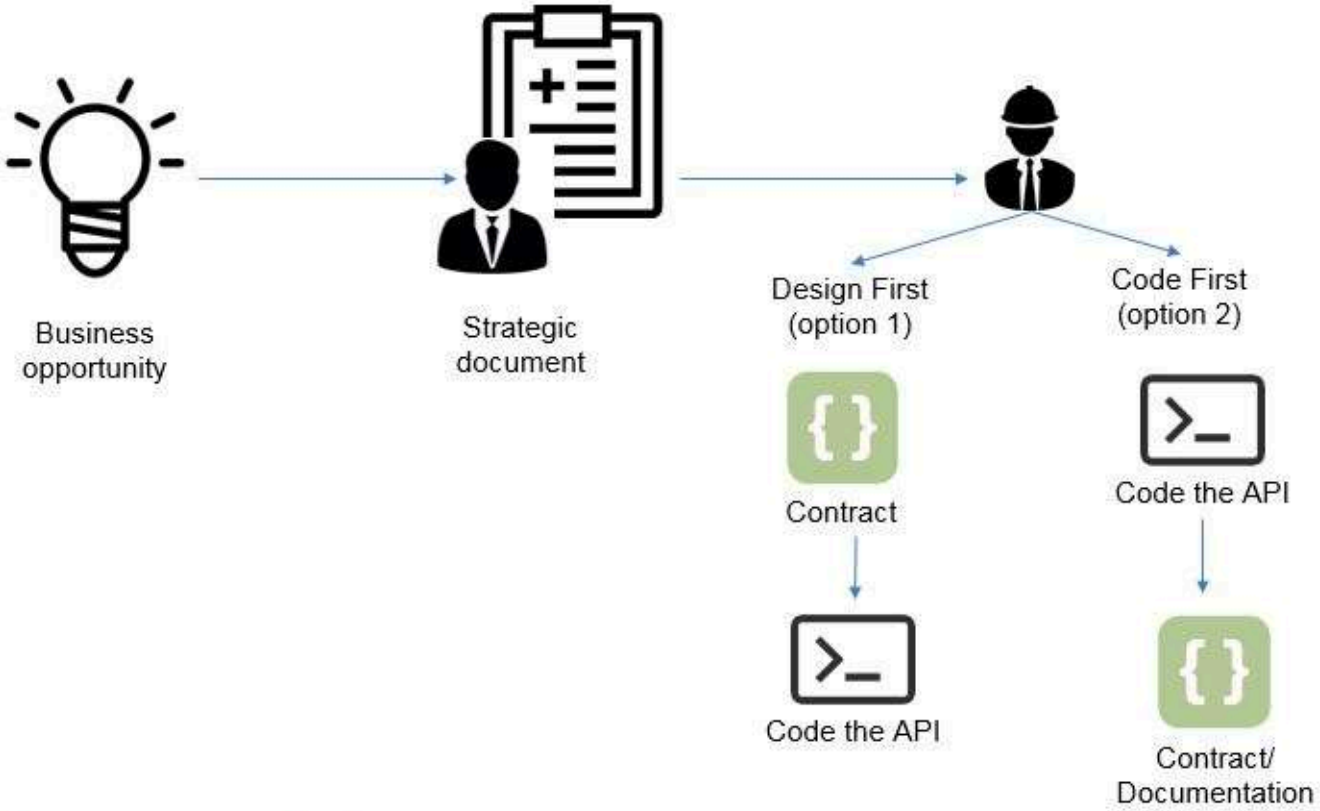


CODE FIRST / DESIGN FIRST



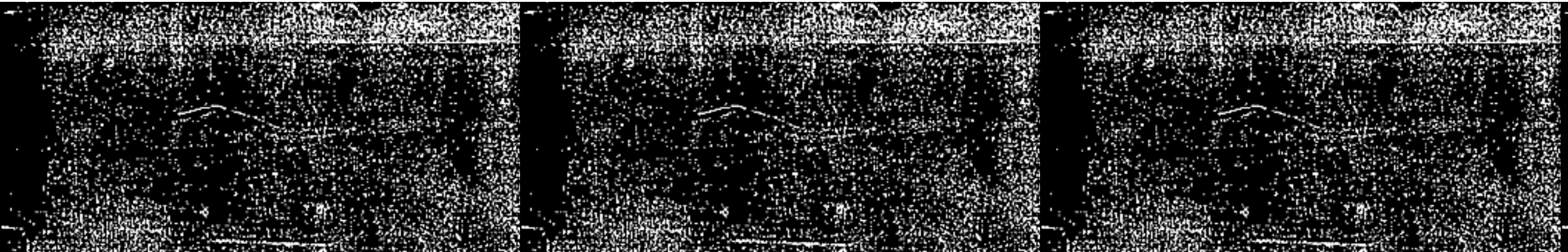
https://swagger.io/blog/api-design/design-first-or-code-first-api-development/

CODE FIRST / DESIGN FIRST

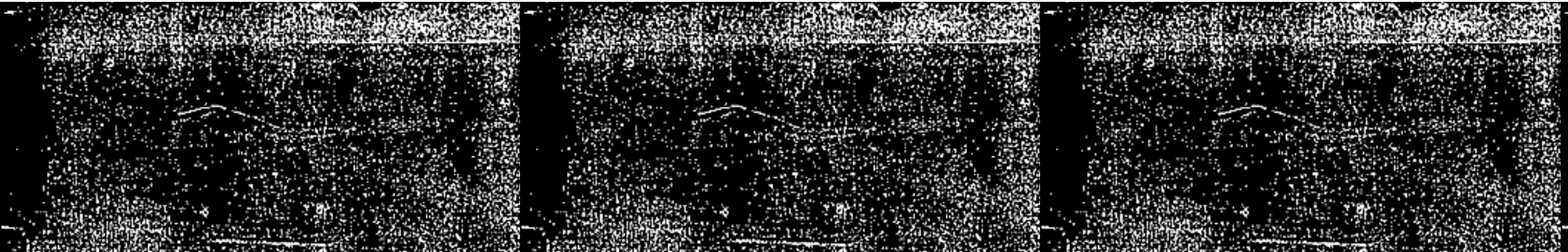


https://swagger.io/blog/api-design/design-first-or-code-first-api-development/

SWAGGER



SWAGGER



LES OUTILS SWAGGER

- **Créer et éditer un fichier de spécification :**
  swagger-editor : https://editor.swagger.io/
- **Générer le code de l'application à partir du fichier de spécifications :**
  swagger-codegen: https://swagger.io/tools/swagger-codegen/
- **Visualiser le fichier de spécification dans une GUI :**
  https://swagger.io/tools/swagger-ui/
- …

LES OUTILS SWAGGER

- 创建和编辑规格文件：
  swagger编辑器：https://editorswagger.io/
- 从规范文件生成应用程序代码：使用 swagger-codegen 工具：https://swagger.io/tools/swagger-codegen/
- 在图形用户界面中查看规范文件：https://swagger.io/tools/swagger-ui/
- …

LES OUTILS SWAGGER

- **Créer et éditer un fichier de spécification :**
  swagger-editor : https://editor.swagger.io/

Swagger Editor
Supported by SMARTBEAR
File ▾ Edit ▾ Insert ▾ Generate Server ▾ Generate Client ▾

```yaml
openapi: 3.0.2
info:
  version: '1.0'
  title: Hello world API
servers:
  - url: http://localhost:8080/
paths:
  /hello:
    get:
      summary: Returns a Helloworld message
      description: optional description
      responses:
        '200':
          description: OK
          content:
            text/plain:
              schema:
                type: string
                example: "Hello Roy !"
```

Hello world API 1.0 OAS3

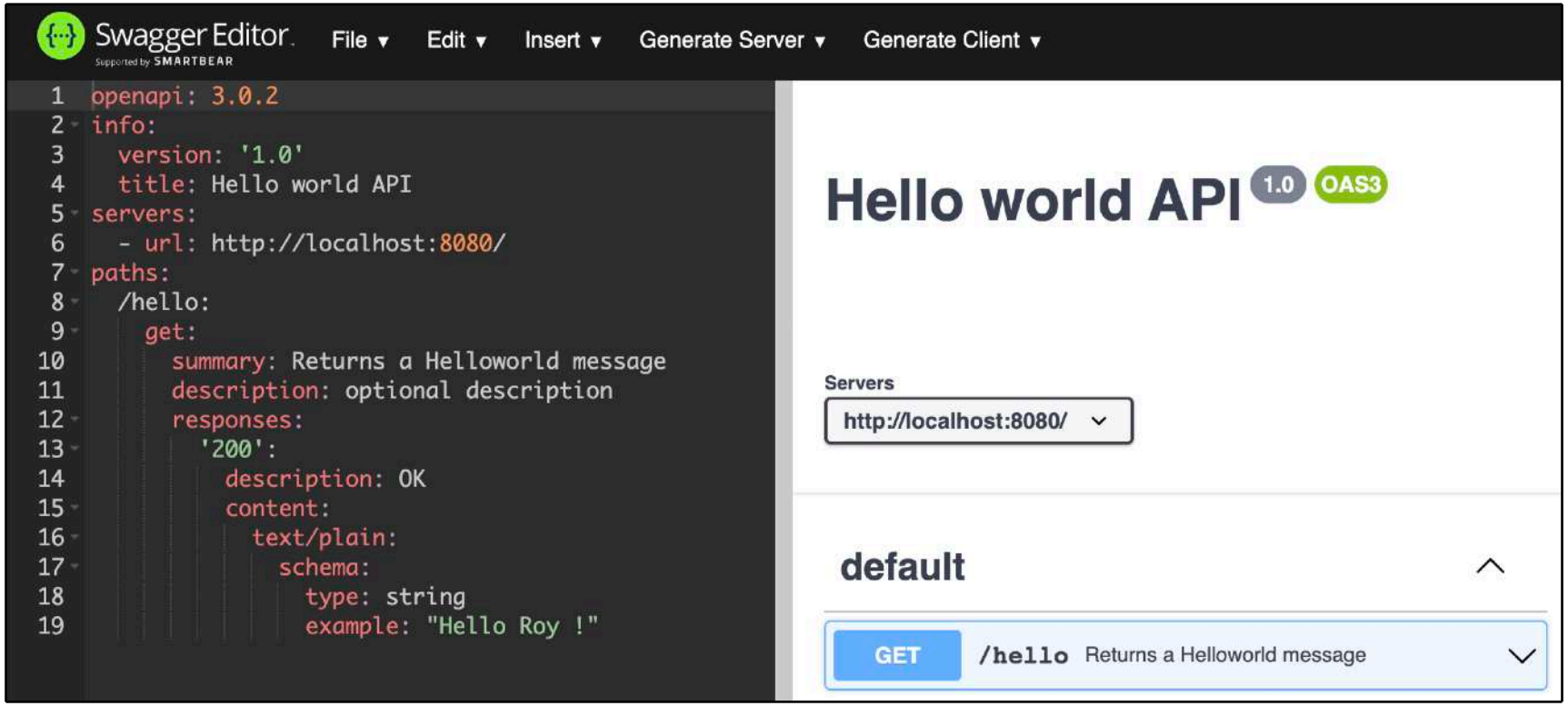
Servers
http://localhost:8080/

default

GET /hello Returns a Helloworld message

LES OUTILS SWAGGER

- 创建和编辑规格文件：
  swagger编辑器：https://editorswagger.io/



LES OUTILS SWAGGER

- **Générer le code de l'application à partir du fichier de spécifications :** swagger-codegen: https://swagger.io/tools/swagger-codegen/

Swagger Editor
Supported by SMARTBEAR

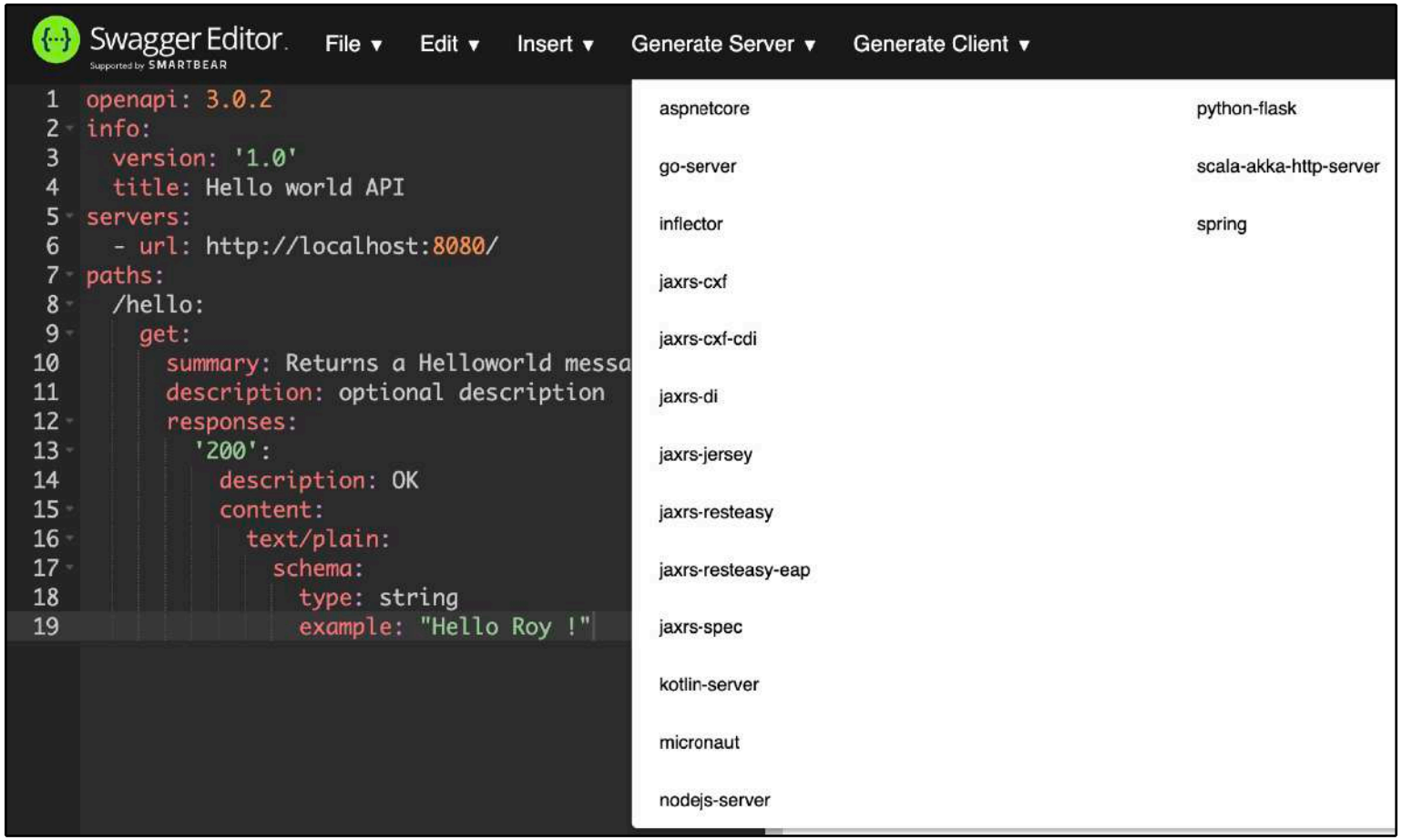
File ▾ Edit ▾ Insert ▾ Generate Server ▾ Generate Client ▾

```
1  openapi: 3.0.2
2  info:
3    version: '1.0'
4    title: Hello world API
5  servers:
6    - url: http://localhost:8080/
7  paths:
8    /hello:
9      get:
10       summary: Returns a Helloworld messa
11       description: optional description
12       responses:
13         '200':
14           description: OK
15           content:
16             text/plain:
17               schema:
18                 type: string
19                 example: "Hello Roy !"
```

aspnetcore
go-server
inflector
jaxrs-cxf
jaxrs-cxf-cdi
jaxrs-di
jaxrs-jersey
jaxrs-resteasy
jaxrs-resteasy-eap
jaxrs-spec
kotlin-server
micronaut
nodejs-server
python-flask
scala-akka-http-server
spring

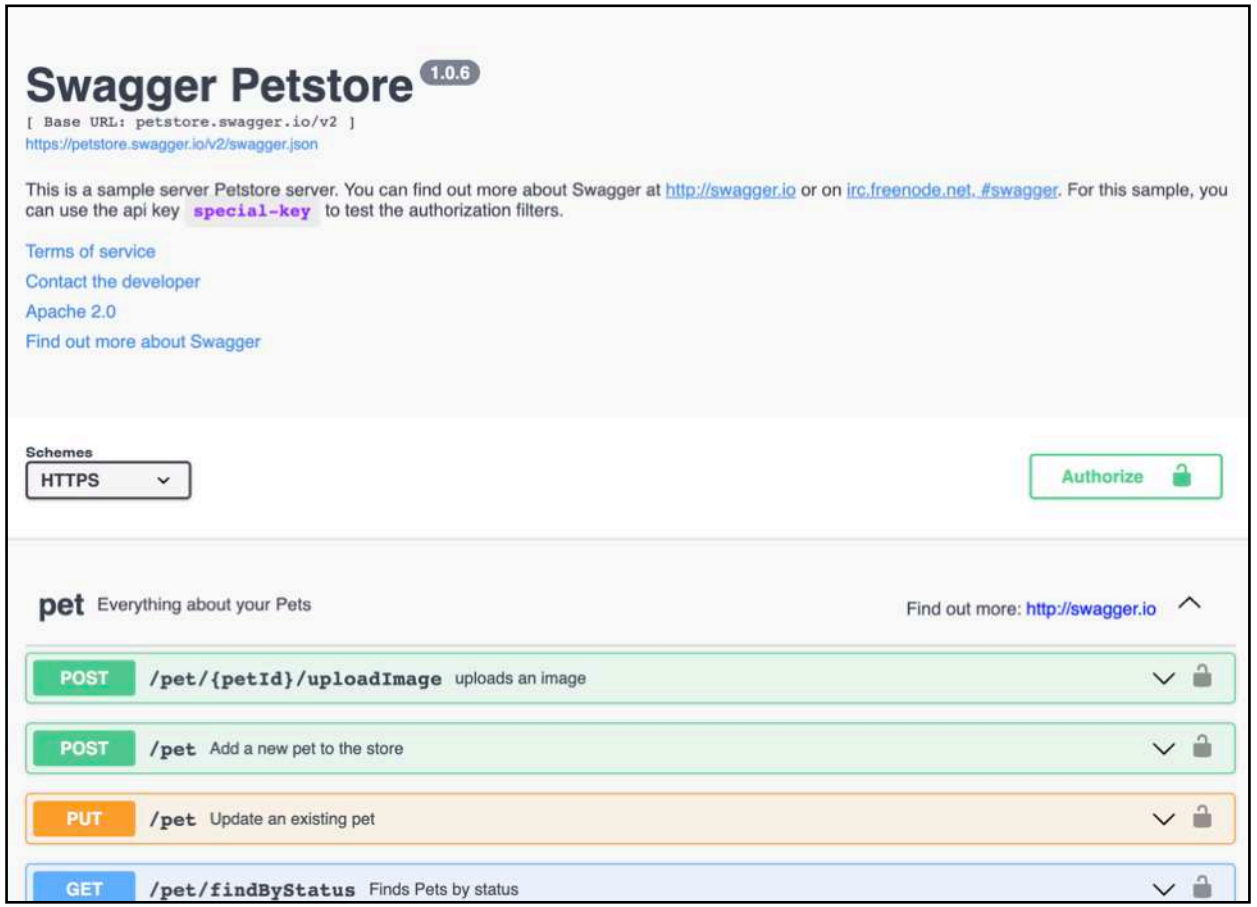
LES OUTILS SWAGGER

- 从规范文件生成应用程序代码：使用 swagger-codegen 工具：https://swagger.io/tools/swagger-codegen/



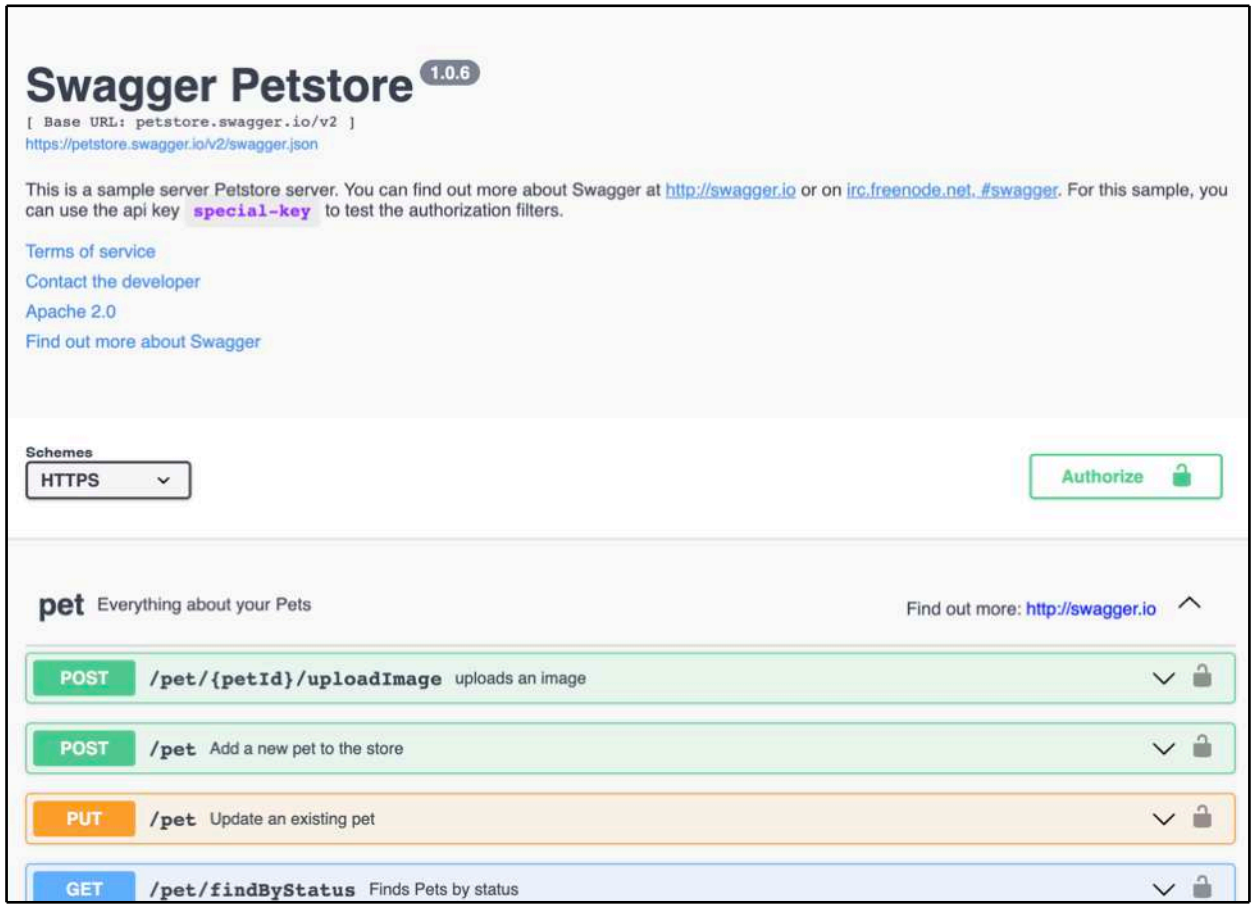
LES OUTILS SWAGGER

- **Visualiser le fichier de spécification dans une GUI :** https://swagger.io/tools/swagger-ui/

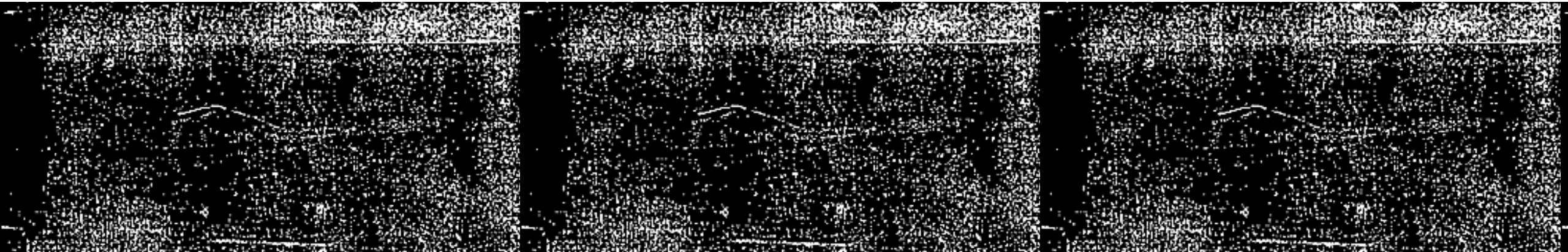


LES OUTILS SWAGGER

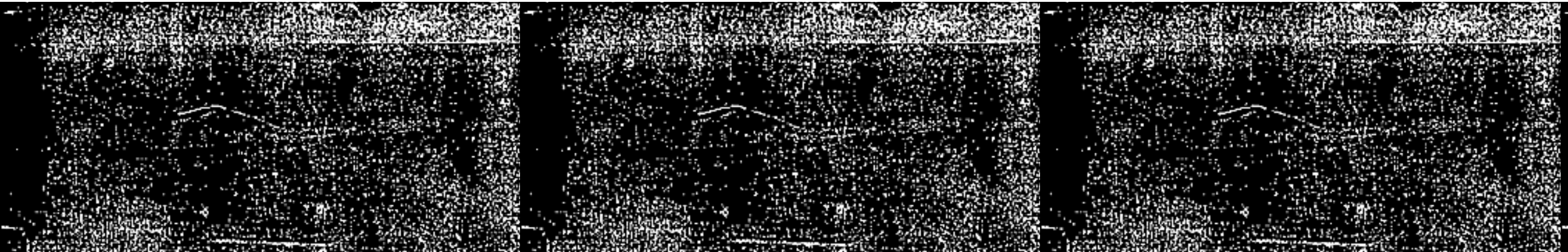
- 在图形用户界面（GUI）中查看规格文件：
  https://swagger.io/tools/swagger-ui/



CONSOMMER UNE API



CONSOMMER UNE API



CONSOMMER UNE API DEPUIS SPRING BOOT

- **RestTemplate**
  - Le thread attend la réponse (Synchrone). Plus simple à comprendre et à débogguer.
- **WebClient**
  - Synchrone / réactif. Idéal pour les hautes charges.

CONSOMMER UNE API DEPUIS SPRING BOOT

- 剩余模板
  - 该线程采用同步模式等待响应，这种设计更易于理解和调试。
- Web客户端
  - 同步式/反应式。特别适用于高负载工况。

LES APPROCHES POUR CONSOMMER UNE API

- **RestTemplate**
  - Le thread attend la réponse (Synchrone). Plus simple à comprendre et à débogguer.
  - RestTemplate n'évolue plus.
  - Dépendance : spring-web
- **WebClient**
  - Synchrone / réactif. Idéal pour les hautes charges.
  - Version recommandée et développée.
  - Dépendance : spring-webflux
- **HttpClient**
  - API intégrée dans Java (depuis 11). Synchrone / Asynchrone.

LES APPROCHES POUR CONSOMMER UNE API

- 剩余模板
  - 该线程采用同步模式等待响应，这种设计更易于理解和调试。
  - **RestTemplate**已停止更新。
  - 依赖性：弹簧网结构
- **Web**客户端
  - 同步式/反应式。特别适用于高负载工况。
  - 推荐版本及开发版本
  - 依赖性：弹簧网通量
- 客户端
  - **Java**内置**API**（自**11**版本起支持）。同步/异步。

RESTEMPLATE

- Déclarer le Bean
  - Créer une configuration pour instancier RestTemplate en tant que Bean Spring.
- Définir le DTO (Data Transfert Object)
  - Créer une classe Java qui correspond à la structure JSON retournée par l'API.
- Création d’un service : Appel GET simple
  - Injecter le RestTemplate et utiliser getForObject ou getForEntity.
- Utiliser dans un Contrôler

RESTEMPLATE

- 声明 Bean
  - 创建配置以将RestTemplate实例化为Spring Bean。
- 定义DTO（数据传输对象）
  - 创建一个与API返回的JSON结构相对应的Java类。
- 创建服务：简单GET请求
  - 注入RestTemplate后，使用getForObject或getForEntity方法。
- 用于控制器中

INSTANCIER UN RESTTEMPLATE

- 1. Déclarer le Bean
  - Créer une configuration pour instancier RestTemplate en tant que Bean Spring.

```
@Configuration
public class AppConfig {

    @Bean
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }

}
```

INSTANCIER UN RESTTEMPLATE

- 1. 宣布 Bean
  - 创建配置以将RestTemplate实例化为Spring Bean。

```
@配置
public class AppConfig {

    @贝恩
    public RestTemplate restTemplate() {
        return new RestTemplate();
    }

}
```

DÉFINIR LE DTO (DATA TRANSFERT OBJECT)

- Créer une classe Java qui correspond à la structure JSON retournée par l'API, ou au moins aux données que l'ont souhaite récupérer.

```
public record UserDto(
        Integer id,
        String  name,
        String  email
    ) {}
```

DÉFINIR LE DTO (DATA TRANSFERT OBJECT)

- 创建一个Java类，用于对应API返回的JSON结构，或至少对应您需要获取的数据类型。

```
公共记录UserDto（整数id,
                字符串name,
                字符串电子邮件
    ）{}
```

CRÉATION D'UN SERVICE : APPEL GET SIMPLE

- Injecter le RestTemplate et utiliser getForObject ou getForEntity

```
@Service
public class UserService {

    private final RestTemplate restTemplate;
    private static final String BASE_URL = "https://jsonplaceholder.typicode.com";

    public UserService(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }

    // GET — objet unique
    public UserDto getUser(int id) {
        String url = BASE_URL + "/users/" + id;
        return restTemplate.getForObject(url, UserDto.class);
    }
}
```

CRÉATION D'UN SERVICE : APPEL GET SIMPLE

- 注入 **RestTemplate** 并
  使用 **getForObject** 或 **getForEntity** 方法

```
@服务
public class UserService {

    私有最终RestTemplate restTemplate;
    private static final String BASE_URL = "https://jsonplaceholder.typicode.com";

    public UserService(RestTemplate restTemplate) {

        这. restTemplate =restTemplate;  }


    // GET – 唯一对象
    public UserDto getUser(int id) {
        String url = BASE_URL + "/users/" + id;
        return restTemplate.getForObject(url, UserDto.class);
    }
}
```

CRÉATION D'UN SERVICE : APPEL GET SIMPLE

```java
@Service
public class UserService {

    private final RestTemplate restTemplate;
    private static final String BASE_URL = "https://jsonplaceholder.typicode.com";

    public UserService(RestTemplate restTemplate) {
        this.restTemplate = restTemplate;
    }

    // GET — objet unique

    // GET — liste
    public List<UserDto> getAllUsers() {
        String url = BASE_URL + "/users";
        UserDto[] arr = restTemplate.getForObject(url, UserDto[].class);
        return arr != null ? List.of(arr) : List.of();
    }

    // POST
    public UserDto createUser(UserDto user) {
        String url = BASE_URL + "/users";
        return restTemplate.postForObject(url, user, UserDto.class);
    }

}
```

CRÉATION D'UN SERVICE : APPEL GET SIMPLE

```
@服务
public class UserService {

    私有最终RestTemplate restTemplate;
    private static final String BASE_URL = "https://jsonplaceholder.typicode.com";

    public UserService(RestTemplate restTemplate) {

        这. restTemplate =restTemplate; }


    // GET – 唯一对象

    // 获取列表
    public List<UserDto> getAllUsers() {
        String url = BASE_URL + "/users";
        UserDto[] arr = restTemplate.getForObject(url, UserDto[].class);
        return arr != null ? List.of(arr) : List.of();
    }

    // POST
    public UserDto createUser(UserDto user) {
        String url = BASE_URL + "/users";
        return restTemplate.postForObject(url, user, UserDto.class);
    }

}
```

UTILISER DANS UN CONTRÔLER

- Injecter le service (comme pour les Bases de données)

```
@RestController
public class UserController {

    @Autowired
    UserService userService;

    @RequestMapping("/api/user2")
    public String welcome() {
        UserDto user = userService.getUser(2);

        return "Hello World GET " + user.email() + " " + user.name();
    }
}
```

UTILISER DANS UN CONTRÔLER

- 注入服务（如数据库）

```
@休息控制器
public class UserController {

    @自动连线
    用户服务用户服务;

    @RequestMapping("/api/user2")
    public String welcome() {
        UserDto user = userService.getUser(2);

        return "Hello World GET " + user.email() + " " + user.name();
    }
}
```

WEBCLIENT

- Construire le WebClient
  - Créer une configuration pour instancier le Webclient.
- Définir le DTO (Data Transfert Object)
  - [idem]
- Création d'un service : Appel GET simple
- Utiliser dans un Contrôler

WEBCLIENT

- 构建 WebClient
  - 创建用于实例化Webclient的配置。
- 定义DTO（数据传输对象）
  - [同上]
- 创建服务：简单GET请求
- 用于控制器中

INSTANCIER UN WEBCLIENT

- 1. Déclarer le Bean
  - Créer une configuration pour instancier WebClient.
  - Déclare directement l'URL de l'API.

```
public class WebClientConfig {

    @Bean
    public WebClient webClient() {
        return WebClient.builder()
            .baseUrl("https://jsonplaceholder.typicode.com")
            .defaultHeader(HttpHeaders.CONTENT_TYPE,
                           MediaType.APPLICATION_JSON_VALUE)
            .build();
    }

}
```

INSTANCIER UN WEBCLIENT

- 1. 宣布 Bean
  - 创建用于实例化WebClient的配置。
  - 直接声明API的URL。

```
public class WebClientConfig {

    @贝恩
    public WebClient webClient() {
        return WebClient.builder()
                .baseUrl("https://jsonplaceholder.typicode.com")
                .defaultHeader(HttpHeaders.CONTENT_TYPE,
                               MediaType.APPLICATION_JSON_VALUE)
                .build();
    }

}
```

DÉFINIR LE DTO (DATA TRANSFERT OBJECT)

- Créer une classe Java qui correspond à la structure JSON retournée par l'API, ou au moins aux données que l'ont souhaite récupérer.

```
public record UserDto(
        Integer id,
        String  name,
        String  email
    ) {}
```

DÉFINIR LE DTO (DATA TRANSFERT OBJECT)

- 创建一个Java类，用于对应API返回的JSON结构，或至少对应您需要获取的数据类型。

```
公共记录UserDto（整数id,
                字符串name,
                字符串电子邮件
    ）{}
```

CRÉATION D'UN SERVICE : APPEL GET SIMPLE

```java
@Service
public class UserReactService {

    private final WebClient webClient;

    public UserReactService(WebClient webClient) {
        this.webClient = webClient;
    }

    // ① Réactif — retourne un Mono (0 ou 1 élément)
    public Mono<UserDto> getUserReactive(int id) {
        return webClient.get()
            .uri("/users/{id}", id)
            .retrieve()
            .onStatus(
                status -> status.is4xxClientError(),   // Predicate<HttpStatusCode>
                res -> Mono.error(new RuntimeException("Utilisateur introuvable"))
            )
            .bodyToMono(UserDto.class);
    }

    // ② Synchrone — .block() bloque jusqu'à la réponse
    public UserDto getUser(int id) {
        return webClient.get()
            .uri("/users/{id}", id)
            .retrieve()
            .bodyToMono(UserDto.class)
            .block();
    }
}
```

CRÉATION D'UN SERVICE : APPEL GET SIMPLE

```
@服务
public class UserReactService {

    私有最终WebClient webClient;

    public UserReactService(WebClient webClient) {
        这。webClient =webClient;
    }

    // ① 反应物 – 返回一个Mono对象（0或1个元素）
    public Mono<UserDto> getUserReactive(int id) {
        return webClient.get()
            .uri("/users/{id}", id)
            .retrieve()
            .onStatus(
                status -> status.is4xxClientError(),   // Predicate<HttpStatusCode>
                res -> Mono.error(new RuntimeException("Utilisateur introuvable"))
            )
            .bodyToMono(UserDto.class);
    }

    // ② 同步模式 – .block() 会一直阻塞直到收到响应
    public UserDto getUser(int id) {
        return webClient.get()
            .uri("/users/{id}", id)
            .retrieve()
            .bodyToMono(UserDto.class)
            .block();
    }
}
```

CRÉATION D'UN SERVICE

```java
@Service
public class UserReactService {

    private final WebClient webClient;

    public UserReactService(WebClient webClient) {
        this.webClient = webClient;
    }

    // ③ Liste — Flux (0 à N éléments)
    public Flux<UserDto> getAllUsersReactive() {
        return webClient.get()
            .uri("/users")
            .retrieve()
            .bodyToFlux(UserDto.class);
    }

    // ④ POST avec body
    public Mono<UserDto> createUser(UserDto user) {
        return webClient.post()
            .uri("/users")
            .bodyValue(user)
            .retrieve()
            .bodyToMono(UserDto.class);
    }

}
```

CRÉATION D'UN SERVICE

```
@服务
public class UserReactService {

    private final WebClient webClient;

    public UserReactService(WebClient webClient) {
        this.webClient = webClient;
    }

    // ③ 列表 – 流（0至N个元素）
    public Flux<UserDto> getAllUsersReactive() {
        return webClient.get()
            .uri("/users")
            .retrieve()
            .bodyToFlux(UserDto.class);
    }

    // ④ POST请求（含body内容）
    public Mono<UserDto> createUser(UserDto user) {
        return webClient.post()
            .uri("/users")
            .bodyValue(user)
            .retrieve()
            .bodyToMono(UserDto.class);
    }

}
```

UTILISATION DANS UN CONTRÔLEUR

```
@RestController
public class UserReactController {

    @Autowired
    UserReactService userService;

    @RequestMapping("/api/reactive/user2")
    public String welcome() {
        UserDto user = userService.getUser(2);

        return "Hello World GET " + user.email() + " " + user.name();
    }

    @RequestMapping("/api/reactive/react_user2")
    public Mono<String> welcomeReact() {
       return userService.getUserReactive(2)
                .map(user -> "Hello World GET " + user.email() + " " + user.name());
    }
}
```

UTILISATION DANS UN CONTRÔLEUR

```
@休息控制器
public class UserReactController {

    @自动连线
    UserReactService用户服务;

    @   RequestMapping( "/api/reactive/user2" )public    String
    welcome() {
        UserDto user = userService .getUser(2);


        返回 "Hello World GET "  +用户.电子邮件() +  " "  +用户.姓名(); }


    @RequestMapping("/api/reactive/react_user2")
    public Mono<String> welcomeReact() {
       return userService.getUserReactive(2)
                .map(user -> "Hello World GET " + user.email() + " " + user.name());
    }
}
```

PASSER UN TOKEN/CLÉ D'ACCÈS À UNE API

- Plusieurs méthodes :
  - API Key dans le header
  - Bearer Token (OAuth2 / JWT)

```
@Bean
public WebClient webClient() {
    return WebClient.builder()
        .baseUrl("https://api.exemple.com")
        .defaultHeader("X-API-Key", "votre-clé-ici")
        .build();
}
```

```
@Bean
public WebClient webClient() {
    return WebClient.builder()
        .baseUrl("https://api.exemple.com")
        .defaultHeader(HttpHeaders.AUTHORIZATION, "Bearer " + token)
        .build();
}
```

PASSER UN TOKEN/CLÉ D'ACCÈS À UNE API

- 多种方法:
  - 报文头中的API密钥
  - 凭证令牌（OAuth2/JWT）

```
豆类
public WebClient webClient() {
    return WebClient.builder()
        .baseUrl( “https://api.exemple.com” )
        .defaultHeader( “X-API-Key” ,  “votre-clé-ici” )
        .构建 ();
}
```

```
豆类
public WebClient webClient() {
    return WebClient.builder()
        .baseUrl("https://api.exemple.com")
        .defaultHeader(HttpHeaders.AUTHORIZATION, "Bearer " + token)
        .build();
}
```